

# Proyecto Paralela

Oscar Vargas Pabon

Los códigos están en [https://github.com/osvarp/paralelComputing\\_week3](https://github.com/osvarp/paralelComputing_week3)

Bajo la carpeta /project

## Implementacion general

main.cpp

Aquí se realizó el input-output del problema. Se cuadran algunas cosas como los streams de salida para que al simular `std::cout` y `std::cerr` envíen los datos a archivos.

ecosystem\_mod.h

Aquí implemento la clase en donde se encapsulara la simulación del ecosistema. En este archivo se definen los métodos que ayudan a ingresar los datos del ecosistema, los métodos que permiten imprimir el estado del sistema, y el método que simulara el ecosistema a alto nivel. Con esto último me refiero a que como el ecosistema en el tiempo  $i$  ( $E^{(i)}$ ) depende enteramente del ecosistema en el tiempo  $i-1$  ( $E^{(i-1)}$ ), entonces solo se podrá paralelizar el paso de  $E^{(i-1)} \rightarrow E^{(i)}$  (igual debo iterar en el tiempo de manera lineal).

Dejo por definir el método 'void Eco::simulate\_1\_step(int iter)', pues es este en el que probaré varios métodos (paralelos y lineal). La definición de este método se encuentra en varios archivos:

ecosystem\_mod\_linear.cpp | Implementacion 1

ecosystem\_mod\_omp.cpp | Implementacion 2

ecosystem\_mod\_loc.cpp | Implementacion 3

Salidas

Explicado utilizando el caso input5x5

A 'standard error' (`std::cerr`) sale el tiempo total contado en milisegundos. Este tiempo lo que cuenta es el tiempo que tomo en ejecutar 'void Eco::simulate(int n\_gen)'. Este tiempo incluye los archivos '\_sim.txt' y '\_log.txt' que se generan ahí.

```
C:\Users\oscar\Local\ponti\7\paralela\project>exec < casefiles\input5x5 > res.txt
Total time(counting I/O stuff) 4
```

A un archivo "\_sim.txt" sale la simulación de cada una de las generaciones con el formato mostrado en el .pdf del proyecto

```
C:\Users\oscar\Local\ponti\7\paralela\project>cat file_sim.txt
```

```
Generation 0
```

```
-----  
|* R F| |* 0 0| |* R 0|  
|F   F| |0   0| |0   0|  
|   *| |   *| |   *|  
|R   | |0   | |R   |  
|R   F| |0   0| |R   0|  
-----
```

```
Generation 1
```

```
-----  
|*R F| |*1 1| |*R 1|  
|   F| |   1| |   1|  
-----
```

A 'standard output' sale la última generación del ecosistema, con el mismo formato de entrada.

```
C:\Users\oscar\Local\ponti\7\paralela\project>cat res.txt
```

```
2 4 3 0 5 5 5
```

```
ROCK 0 0
```

```
ROCK 2 4
```

```
RABBIT 3 0
```

```
FOX 4 0
```

```
FOX 4 3
```

A un archivo '\_log.txt' sale información/estadísticas de la simulación

```
C:\Users\oscar\Local\ponti\7\paralela\project>cat file_log.txt
```

```
Iteration 0
```

```
Time -1
```

```
FrequencyElement 3 RABBIT
```

```
FrequencyElement 4 FOX
```

```
FrequencyElement 2 ROCK
```

```
FrequencyElement 16 NONE
```

```
Iteration 1
```

```
Time 0
```

```
FrequencyElement 3 RABBIT
```

```
FrequencyElement 4 FOX
```

```
FrequencyElement 2 ROCK
```

```
FrequencyElement 16 NONE
```

```

FrequencyElement 2 ROCK
FrequencyElement 20 NONE

Iteration 6
Time 0
FrequencyElement 1 RABBIT
FrequencyElement 2 FOX
FrequencyElement 2 ROCK
FrequencyElement 20 NONE

TotalTime 0

Remember the iteration index is 0-indexed and
the time is given in milliseconds

```

Los tiempos de cada iteración están contados en milisegundos y ‘TotalTime’ es la suma de todos los tiempos de cada iteración. Notar que los tiempos solo consideran el tiempo que toma ejecutar ‘void Eco::simulate\_1\_step(int iter)’

## Implementacion 1 (lineal)

Implementada en ‘ecosystem\_mod\_linear.cpp’. Separo la parte en donde muevo las ‘rabbits’ y los ‘foxes’. Cada una de las dos partes tiene, a su vez, dos etapas.

La primera revisa cada ‘rabbit’ a que casilla va a saltar, según las reglas de movimiento. Esta información la guarda luego en el vector ‘rab’

```

std::vector<std::vector<int>>> rab;
for(int i=0;i<n;++i)for(int j=0;j<m;++j)if(grid[i][j]==RABBIT){
    std::vector<int> pos;
    for(int mind=0;mind<int(mov.size());++mind){
        int ni=i+mov[mind][SD_CONST],nj=j+mov[mind][SD_CONST^1];

        // Note: possible ambiguity on the rules here if the 'unmoved offspring'
        if(ni>=0&&nj>=0&&ni<n&&nj<m&&grid[ni][nj]==NONE) pos.push_back(mind);
    }
    int mind=(pos.empty())?-1:pos[(i+j+iter+IND_CONST)%pos.size()+pos.size()%pos.size()];
    rab.push_back({i,j,proc[i][j]+1,mind});
}

```

La segunda mueve cada ‘rabbit’ y gestiona cosas como si tienen hijos o colisiones

```

for(const std::vector<int> &a_rab:rab){
    int i=a_rab[0],j=a_rab[1],tproc=a_rab[2],mind=a_rab[3];

    if(mind==-1){proc[i][j]=tproc;continue;}//Stupid untold rule

    if(tproc>GEN_PROC_RABBITS)tproc=proc[i][j]=0;
    else grid[i][j]=proc[i][j]=NONE;

    int ni=i+mov[mind][SD_CONST],nj=j+mov[mind][SD_CONST^1];
    if(grid[ni][nj]==NONE)grid[ni][nj]=RABBIT,proc[ni][nj]=-1;
    if(proc[ni][nj] < tproc ) proc[ni][nj]=tproc;
}

```

Los movimientos de los 'foxes' son iguales, con la única diferencia de que ahora tengo que revisar el asunto de la 'food' y las posibles muertes que resulten de eso (por 'starving' o 'eating'). Sin embargo, el esquema general es idéntico a el de los 'rabbits', solo que con un par de datos y condiciones extra.

### Implementacion 2 (OpenMP)

Implementada en 'ecosystem\_mod\_omp.cpp'. Tome la implementación lineal y paralelize todos los ciclos con '#pragma omp paralelize for'

```
// move rabbits
std::vector<std::vector<int>>> rab;
#pragma omp paralelize for
for(int i=0;i<n;++i){
#pragma omp paralelize for
for(int j=0;j<m;++j) if(grid[i][j]==RABBIT){
```

Esto, sin embargo, genera un par de 'race-conditions'. El primero de estos se ve cuando estoy tratando de insertar elementos al vector 'rab' (igual para los 'foxes' se genera este problema), pues 'push\_back()' no es 'thread safe'. Esto se solucionó con '#pragma omp critical'

```
#pragma omp critical
rab.push_back({i,j,proc[i][j]+1, mind});
```

El segundo de estos se da cuando hay varias 'rabbit' (y lo mismo para los 'foxes') que terminan saltando a la misma casilla, en cuyo caso debo dejar la que tiene mayor tiempo de procreación. Esto genera, tambien, una 'race-condition' sobre los valores de la casilla a la que van a salter. Esto también se resolvió con '#pragma omp critical'.

```
#pragma omp critical
{ // there are surely better ways to ensure paralelism here
if(grid[ni][nj]==NONE)grid[ni][nj]=RABBIT,proc[ni][nj]=-1;
if(proc[ni][nj] < tproc ) proc[ni][nj]=tproc;
}
```

### Implementación 3 (OpenMP con Locks)

Implementada en 'ecosystem\_mod\_loc.cpp'. Tome la implementación anterior (OpenMP) y me preocupó que la solución a los 'race\_condition' utilizando '#pragma omp critical' pudiera afectar los tiempos de ejecución. La idea de eso es que el primer problema es fácilmente solucionable si no hago 'push\_back()', lo cual se puede solucionar creando memoria de mas para que cada una de estas iteraciones trabaje en una posición distinta. El segundo problema se puede solucionar con 'locks', lo cual tiene sentido porque los únicos 'race-condition' que se generan es cuando varias 'rabbit' ('foxes') aledañas van a saltar a la misma casilla. Para esto se generó

```
static std::vector<std::vector<omp_lock_t>> lock(n,std::vector<omp_lock_t>(m));
for(int i=0;i<n;++i)for(int j=0;j<m;++j) omp_init_lock(&lock[i][j]);
```

Y se implementa el 'lock' como

```
omp_set_lock(&lock[ni][nj]);
if(grid[ni][nj]==NONE)grid[ni][nj]=RABBIT,proc[ni][nj]=-1;
if(proc[ni][nj] < tproc ) proc[ni][nj]=tproc;
omp_unset_lock(&lock[ni][nj]);
```

### Implementación 3 (OpenMP sin Locks)

Implementada en 'ecosystem\_mod\_nlc.cpp'. Note que si, en la segunda etapa, en vez de enviar el 'rabbit'(o 'fox') de la casilla  $(i, j) \rightarrow (ni, nj)$ , revisamos cuales 'rabbits' van a saltar a nuestra casilla, entonces podemos saltarnos el paso de sincronización sobre el que estaba usando los 'locks'

```
for(const std::vector<int> &mv:mov ){
    int ni=i+mv[SD_CONST],nj=j+mv[SD_CONST^1];
    if(ni<0||nj<0||ni>n||nj>m||rab[ni][nj].empty()||rab[ni][nj][1]==-1)continue;
    std::vector<int> &a_rab=rab[ni][nj];
    int tproc=a_rab[0],mind=a_rab[1];if(tproc>GEN_PROC_RABBITS)tproc=0;
    int nni=ni+mv[mind][SD_CONST],nnj=nj+mv[mind][SD_CONST^1];
    if(nni==i&&nnj==j){
        if(grid[i][j]==NONE)grid[i][j]=RABBIT,proc[i][j]=-1;
        if(proc[i][j] < tproc ) proc[i][j]=tproc;
    }
}
```

También hice unos truquitos para evitar intentar saltar desde todos los nodos.

### Casos Comparacion

Input100x100

Corremos las 4 versiones en este caso

```
C:\Users\oscar\Local\ponti\7\paralela\project>..\paral.cmd
japeto@61c7637962b3:~/app$ g++ -fopenmp main.cpp ecosystem_mod_linear.cpp -o lin.out
japeto@61c7637962b3:~/app$ ./lin.out < casefiles/input100x100 > lin_res.txt
Total time(counting I/O stuff) 157881
japeto@61c7637962b3:~/app$ g++ -fopenmp main.cpp ecosystem_mod_omp.cpp -o omp.out
japeto@61c7637962b3:~/app$ g++ -fopenmp main.cpp ecosystem_mod_omp.cpp -o omp.out
japeto@61c7637962b3:~/app$ ./omp.out < casefiles/input100x100 > omp_res.txt
Total time(counting I/O stuff) 164479
japeto@61c7637962b3:~/app$ g++ -fopenmp main.cpp ecosystem_mod_loc.cpp -o loc.out
japeto@61c7637962b3:~/app$ ./loc.out < casefiles/input100x100 > loc_res.txt
Total time(counting I/O stuff) 196355
japeto@61c7637962b3:~/app$
```

```
japeto@4c2116c69b10:~/app$ g++ -fopenmp main.cpp ecosystem_mod_nlc.cpp -o nlc.out
japeto@4c2116c69b10:~/app$ ./nlc.out < casefiles/input100x100 > nlc_res.txt
Total time(counting I/O stuff) 224477
```

Y verificamos que halla simulado bien

```
C:\Users\oscar\Local\ponti\7\paralela\project>fc loc_res.txt casefiles\output100x100
Comparando archivos loc_res.txt y CASEFILES\OUTPUT100X100
FC: no se han encontrado diferencias
```

```
C:\Users\oscar\Local\ponti\7\paralela\project>fc omp_res.txt casefiles\output100x100
Comparando archivos omp_res.txt y CASEFILES\OUTPUT100X100
FC: no se han encontrado diferencias
```

```
C:\Users\oscar\Local\ponti\7\paralela\project>fc lin_res.txt casefiles\output100x100
Comparando archivos lin_res.txt y CASEFILES\OUTPUT100X100
FC: no se han encontrado diferencias
```

```
C:\Users\oscar\Local\ponti\7\paralela\project>fc lin_sim.txt omp_sim.txt
Comparando archivos lin_sim.txt y OMP_SIM.TXT
FC: no se han encontrado diferencias
```

```
C:\Users\oscar\Local\ponti\7\paralela\project>fc lin_sim.txt loc_sim.txt
Comparando archivos lin_sim.txt y LOC_SIM.TXT
FC: no se han encontrado diferencias
```

```
C:\Users\oscar\Local\ponti\7\paralela\project>fc nlc_res.txt casefiles\output100x100
Comparando archivos nlc_res.txt y CASEFILES\OUTPUT100X100
FC: no se han encontrado diferencias
```

```
C:\Users\oscar\Local\ponti\7\paralela\project>fc nlc_sim.txt omp_sim.txt
Comparando archivos nlc_sim.txt y OMP_SIM.TXT
FC: no se han encontrado diferencias
```

Calculamos speedups  $Speedup = \frac{Tiempo\ lineal}{Tiempo\ paralelo}$

|              | Lin    | omp           | loc          | nlc          |
|--------------|--------|---------------|--------------|--------------|
| TiempoIO     | 157881 | 164479        | 196355       | 224477       |
| TiempoSinIO  | 48753  | 50182         | 79512        | 111569       |
| SpeedupIO    | 1      | 0.95988545650 | 0.8040589748 | 0.7033281806 |
| speedupSinIO | 1      | 0.9715236     | 0.613152731  | 0.436976220  |

Notar que estos valores pueden haberse visto afectados por las condiciones de mi maquina.  
En una previa ocasión obtuve los tiempos con IO

lin 157546

omp 155463

loc 183765

Los archivos “\_log.txt” están en /outFiles/100

Input200x200

```
japeto@61c7637962b3:~/app$ ./lin.out < casefiles/input200x200 > lin_res.txt
./omp.out < casefiles/input200x200 > omp_res.txt
./loc.out < casefiles/input200x200 > loc_res.txt
./nlc.out < casefiles/input200x200 > nlc_res.txt
Total time(counting I/O stuff) 404354
japeto@61c7637962b3:~/app$ ./omp.out < casefiles/input200x200 > omp_res.txt
Total time(counting I/O stuff) 404803
japeto@61c7637962b3:~/app$ ./loc.out < casefiles/input200x200 > loc_res.txt
Total time(counting I/O stuff) 518510
japeto@61c7637962b3:~/app$ ./nlc.out < casefiles/input200x200 > nlc_res.txt
Total time(counting I/O stuff) 657803
japeto@61c7637962b3:~/app$
```

Verificamos que el código sea correcto

```
C:\Users\oscar\Local\ponti\7\paralela\project>fc lin_res.txt casefiles\output200x200
Comparando archivos lin_res.txt y CASEFILES\OUTPUT200X200
FC: no se han encontrado diferencias

C:\Users\oscar\Local\ponti\7\paralela\project>fc omp_res.txt casefiles\output200x200
Comparando archivos omp_res.txt y CASEFILES\OUTPUT200X200
FC: no se han encontrado diferencias

C:\Users\oscar\Local\ponti\7\paralela\project>fc loc_res.txt casefiles\output200x200
Comparando archivos loc_res.txt y CASEFILES\OUTPUT200X200
FC: no se han encontrado diferencias

C:\Users\oscar\Local\ponti\7\paralela\project>fc nlc_res.txt casefiles\output200x200
Comparando archivos nlc_res.txt y CASEFILES\OUTPUT200X200
FC: no se han encontrado diferencias
```

```
C:\Users\oscar\Local\ponti\7\paralela\project>fc lin_sim.txt omp_sim.txt
Comparando archivos lin_sim.txt y OMP_SIM.TXT
FC: no se han encontrado diferencias

C:\Users\oscar\Local\ponti\7\paralela\project>fc lin_sim.txt loc_sim.txt
Comparando archivos lin_sim.txt y LOC_SIM.TXT
FC: no se han encontrado diferencias

C:\Users\oscar\Local\ponti\7\paralela\project>fc lin_sim.txt nlc_sim.txt
Comparando archivos lin_sim.txt y NLC_SIM.TXT
FC: no se han encontrado diferencias
```

Calculamos speedups  $Speedup = \frac{Tiempo\ linear}{Tiempo\ paralelo}$

|             | Lin    | omp         | loc        | nlc         |
|-------------|--------|-------------|------------|-------------|
| TiempoIO    | 404354 | 404803      | 518510     | 657803      |
| TiempoSinIO | 202142 | 202746      | 312466     | 448984      |
| SpeedupIO   | 1      | 0.998890818 | 0.77983838 | 0.614703794 |

|              |   |             |            |            |
|--------------|---|-------------|------------|------------|
| speedupSinIO | 1 | 0.997020903 | 0.64692478 | 0.45022094 |
|--------------|---|-------------|------------|------------|

Los archivos “\_log.txt” están en /outFiles/200